To Cascade or Not to Cascade? An Empirical Study of Two-Stage Safety Detection for Streaming LLM Outputs

JIANMIN QIU, Southeast University, China
JINGUANG HAN, Southeast University, China

Real-time safety detection during LLM output generation is critical for deployed AI systems. A natural approach is confidence-based early exit: monitoring a lightweight classifier at each generation step and stopping when confidence exceeds a threshold. The difficulty is that low thresholds maximize recall but produce unacceptable false positive rates (FPR), while high thresholds miss toxic content.

We investigate whether a heterogeneous two-stage cascade—where the two stages employ models of different architectures (an encoder classifier vs. a generative safety model)—can resolve this tension. Stage 1 uses a fine-tuned RoBERTa with Wald's formula-derived thresholds to flag potentially unsafe prefixes during streaming generation; Stage 2 applies Llama Guard, a fundamentally different and larger safety model, to the flagged prefix to eliminate false alarms. Because the two stages use different models with independent error modes, the cascade does not collapse to a single threshold.

Experiments on four safety benchmarks spanning low-toxicity (CivilComments, 8%), high-toxicity (BeaverTails, 57%; PKU-SafeRLHF, 52%), and adversarial (Qwen3GuardTest, 100%) scenarios show that single-stage streaming achieves 71–96% token savings with competitive F1 (0.356–0.780). The cascade reduces FPR by 50–96% over single-stage streaming, but at the cost of recall: F1 drops by 19–85% because Llama Guard, operating on short prefixes, overrules many correct Stage 1 decisions. We further show that prefix-aware fine-tuning of Llama Guard on short-prefix data improves cascade F1 by 25–209% and reduces prefix failures by 52–79%, partially addressing this bottleneck. However, cascade effectiveness is ultimately bounded by Stage 2's capability ceiling: on CivilComments where Llama Guard achieves only F1=0.138 on full text, the cascade provides no benefit.

These results establish single-stage streaming as a practical approach for efficient safety detection, while revealing that the heterogeneous cascade can hurt rather than help when Stage 2's prefix-level judgment is unreliable.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; *Artificial intelligence*.

Additional Key Words and Phrases: Large Language Models, Safety Detection, Cascade Classification, Early Exit, Llama Guard, Streaming Inference

ACM Reference Format:
Jianmin Qiu and Jinguang Han. 2026. To Cascade or Not to Cascade? An Empirical Study of Two-Stage Safety Detection for Streaming LLM Outputs. *ACM Trans. Intell. Syst. Technol.* 1, 1, Article 1 (April 2026), 22 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Availability of Data and Code

The datasets are publicly available: CivilComments [5], BeaverTails [6], PKU-SafeRLHF [7], and Qwen3GuardTest [8]. Our code and trained classifiers are available at: https://github.com/Qiujianmin/streaming-sprt-safety-detection. The

Authors' Contact Information: Jianmin Qiu, 230239771@seu.edu.cn, Southeast University, Nanjing, China; Jinguang Han, Southeast University, Nanjing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

prefix-fine-tuned QLoRA adapter weights for Llama Guard are available at https://github.com/Qiujianmin/streaming-sprt-safety-detection/tree/main/llama_guard_prefix_adapter.

2 Introduction

2.1 Background and Motivation

Large Language Models (LLMs) are widely deployed, but they can generate toxic content, hate speech, and harmful instructions at scale. Most production systems use post-hoc filtering: wait for complete text generation, then apply a safety classifier to the full output.

This approach has clear limitations: resources are wasted generating unsafe content over hundreds of tokens, users wait for content that is subsequently blocked, and there is no mechanism to stop toxic content generation mid-stream.

A natural alternative is streaming detection: monitor a safety classifier at each generation step and make an early decision. The key design question is *when to stop and what confidence threshold should trigger a decision.*

2.2 The Threshold Selection Problem

Standard practice uses arbitrary confidence thresholds (e.g., 0.90, 0.95) chosen without theoretical justification. The resulting error rates are unpredictable and must be determined empirically for each deployment.

Wald's Sequential Probability Ratio Test (SPRT) [1] provides a classical formula mapping target error rates (α, β) to decision thresholds:

$$A = \log \frac{1-\beta}{\alpha}, \quad B = \log \frac{\beta}{1-\alpha} \tag{1}$$

Although these thresholds were derived for the classical i.i.d. setting, they provide a principled starting point for threshold selection even in our non-i.i.d. setting.

2.3 Contributions

This paper makes the following contributions:

(1) We propose Cascade-Guard, a two-stage architecture combining RoBERTa (Stage 1, fast screening) with Llama Guard (Stage 2, precise confirmation). The two stages use fundamentally different models—a 125M-parameter encoder vs. an 8B-parameter generative safety model—so the cascade does not collapse to a single threshold. The intersection of two independent classifiers' positive regions produces a decision boundary that neither classifier alone can achieve.

(2) We show that Wald's formula with prior adjustment $(\alpha, \beta, \pi) \mapsto A'$ provides a principled, interpretable parameterization for the Stage 1 screening threshold. We explicitly acknowledge that this is mathematically equivalent to confidence thresholding with $\tau = \sigma(A')$. The contribution is not a novel sequential test, but the three-dimensional parameter space (α, β, π) that encodes domain knowledge (target FPR, FNR, toxicity prior) and provides monotonic, predictable control over the precision-recall trade-off (Section 5.5).

(3) We evaluate on four diverse safety benchmarks spanning low-toxicity (CivilComments, 8%), high-toxicity (BeaverTails 57%, PKU-SafeRLHF 52%), and adversarial (Qwen3GuardTest, 100%) scenarios, with full confidence-threshold sweeps ($\tau \in \{0.10, \ldots, 0.95\}$) as baselines. We additionally evaluate a prefix-fine-tuned Llama Guard as Stage 2 to test whether adapting Stage 2 to short prefixes improves cascade performance.

(4) We benchmark against both full-text Llama Guard (precision upper bound) and full-text RoBERTa (speed upper bound), and identify the prefix bottleneck: Llama Guard, trained on complete texts, frequently overrules correct

Stage 1 decisions when applied to short prefixes. Prefix-aware fine-tuning of Stage 2 with QLoRA (40K prefix samples) partially resolves this bottleneck, improving cascade F1 by 25–209% and reducing prefix failures by 52–79% (Section 7.2.4).

3 Related Work

3.1 LLM Safety Detection

Classifier-based methods use fine-tuned models for safety classification, including Llama Guard [9], GPT-4 Moderation API [10], Perspective API [12], and BERT/RoBERTa-based approaches [13, 14] that achieve state-of-the-art performance on hate speech detection. All require full text and lack streaming capability. Our method enables early decisions during generation.

Among LLM-based safety classifiers, Llama Guard 1/2/3 [9, 15] are 7B–8B-parameter models that achieve high accuracy on nuanced safety judgment. Llama Guard 3 8B serves as our Stage 2 classifier. GPT-4 Moderation [10] provides multi-dimensional safety scores across 13 categories.

Our heterogeneous cascade specifically leverages the complementary strengths of fast encoder-based classifiers (RoBERTa, ∼7ms) and precise generative safety models (Llama Guard, ∼86ms), using the former for streaming screening and the latter only when needed for confirmation.

3.2 Sequential Analysis in Machine Learning

Classical SPRT [1] provides the optimal sequential test for i.i.d. observations, minimizing expected sample size under error constraints. Non-i.i.d. extensions include sequential analysis for Markov processes [3] and online learning [4]. Recent ML applications include online LLM text detection [16] and early exit for neural networks [17]. These address different problems and do not handle LLM safety specifically.

3.3 Early Exit and Streaming Inference

Early exit networks [17, 19] exit at intermediate layers for computational savings. Streaming LLMs [18] use efficient attention mechanisms. Neither addresses safety detection. Our work applies early exit principles to the safety domain with principled threshold selection.

3.4 Cascade Classification

Cascade classifiers [20] chain multiple classifiers where early stages reject easy negatives and later stages provide more precise classification. Classical work on face detection by Viola and Jones demonstrated that a cascade of increasingly complex classifiers can achieve both high accuracy and real-time speed. Our cascade architecture applies this principle to streaming safety detection with a key innovation: heterogeneous model stages.

Traditional cascades use the same classifier at different thresholds or context levels, which limits the diversity of decision boundaries. In contrast, Cascade-Guard uses RoBERTa (125M-parameter encoder) for Stage 1 screening and Llama Guard (8B-parameter generative safety model) for Stage 2 confirmation. These models have fundamentally different architectures, training data, and inductive biases, producing independent error modes. This model diversity is the key mechanism that prevents the cascade from collapsing to a single threshold.

Llama Guard is a natural candidate for Stage 2: it is specifically trained for safety classification across multiple harm categories and achieves state-of-the-art precision, but its ~86ms inference cost makes it too expensive for per-token streaming. By invoking it only when Stage 1 flags a prefix, we amortize its cost over the entire streaming pipeline.

4 Problem Formulation

4.1 Streaming Safety Detection

Let $X = (X_1, X_2, \ldots, X_T)$ be the sequence of tokens generated by an LLM. We formulate safety detection as a binary hypothesis testing problem:

- H_0: The content is **safe**
- H_1: The content is **unsafe**

We seek a stopping time τ and decision function $\delta(\tau) \in \{\text{SAFE}, \text{UNSAFE}\}$ that minimize expected detection latency while controlling error rates:

$$\min_{\tau} \mathbb{E}[\tau] \quad \text{subject to} \quad \mathbb{P}^{H_0}(\text{FP}) \leq \alpha, \quad \mathbb{P}^{H_1}(\text{FN}) \leq \beta \tag{2}$$

4.2 Key Challenge: Autoregressive Dependence

LLM tokens violate the i.i.d. assumption: $P(X_t \mid X_{<t}) \neq P(X_t)$. This means the classical Wald SPRT—which assumes independent observations—does not directly apply with its theoretical guarantees intact. We return to this point in Section 6.2.

5 Method: Heterogeneous Two-Stage Cascade

5.1 Threshold Derivation via Wald's Formula

Standard practice for confidence-based early exit selects thresholds τ via grid search or heuristics, yielding no principled relationship between the chosen threshold and the resulting error rates. Wald's classical formula [1] provides a principled mapping from target error rates (α, β) and a domain prior π to concrete thresholds, replacing ad-hoc grid search over $\tau \in [0, 1]$ with an interpretable three-dimensional parameter space.

For i.i.d. observations, Wald defined the cumulative log-likelihood ratio:

$$S_t^{\text{true}} = \sum_{i=1}^{t} \log \frac{P(X_i \mid H_1)}{P(X_i \mid H_0)} \tag{3}$$

and the optimal stopping rule: stop when $S_t^{\text{true}} \notin (B, A)$ where $A = \log \frac{1-\beta}{\alpha}$, $B = \log \frac{\beta}{1-\alpha}$.

Conditional likelihood ratios. For autoregressive sequences, the proper generalization is:

$$S_t^{\text{cond}} = \sum_{i=1}^{t} \log \frac{P(X_i \mid X_{<i}, H_1)}{P(X_i \mid X_{<i}, H_0)} \tag{4}$$

Classifier approximation. We do not have access to the true conditional distributions $P(X_i \mid X_{<i}, H_j)$. Instead, we use a neural classifier f_θ that outputs a toxicity probability for the entire prefix:

$$p_t = f_\theta(x_{\leq t}) \in [0, 1] \tag{5}$$

For a well-calibrated classifier, the logit $\log \frac{p_t}{1-p_t}$ approximates the posterior log-odds of the full prefix:

$$\log \frac{p_t}{1-p_t} \approx \log \frac{P(H_1)}{P(H_0)} + \log \frac{P(x_{\leq t} \mid H_1)}{P(x_{\leq t} \mid H_0)} \tag{6}$$

This means the classifier logit already incorporates *all* preceding context through the Transformer's self-attention. This approximation assumes the classifier is well-calibrated; in practice, we apply temperature scaling (Table 11) to reduce calibration error before using the logit as a decision score. We therefore use the logit *directly* as our evidence score, rather than accumulating over individual token-level likelihood ratios:

$$\tilde{S}_t = \log \frac{\text{clip}(p_t, \varepsilon, 1-\varepsilon)}{1 - \text{clip}(p_t, \varepsilon, 1-\varepsilon)}, \quad \varepsilon = 10^{-8} \tag{7}$$

where $\text{clip}(x, a, b) = \min(\max(x, a), b)$ clamps the probability to $[\varepsilon, 1-\varepsilon]$ for numerical stability, following standard practice in log-odds computation.

Remark 5.1 (Relationship to Confidence Thresholding)**.** The single-stage decision rule $\tilde{S}_t \geq A'$ is equivalent to confidence thresholding $p_t \geq \sigma(A')$. Despite this equivalence, the (α, β, π) parameterization offers concrete advantages over direct threshold selection: (1) decreasing π consistently lowers the UNSAFE threshold and increases recall; (2) practitioners can set α to their maximum tolerable FPR and π to their estimated toxicity rate; (3) the prior-adjusted threshold combines naturally with the cascade's two-model architecture. The value is not in the SPRT algorithm itself (which requires i.i.d. observations we do not have), but in the principled mapping from deployment requirements to thresholds.

5.2 Wald-Derived Decision Thresholds

Given target error rate parameters (α, β), we compute thresholds using Wald's formula:

$$A = \ln \frac{1-\beta}{\alpha}, \quad B = \ln \frac{\beta}{1-\alpha} \tag{8}$$

Decision rule.

$$\delta(\tau) = \begin{cases} \text{UNSAFE} & \text{if } \tilde{S}_t \geq A \\ \text{SAFE} & \text{if } \tilde{S}_t \leq B \\ \text{CONTINUE} & \text{if } B < \tilde{S}_t < A \end{cases} \tag{9}$$

For the standard setting ($\alpha = 0.05, \beta = 0.10$):

- $A = \ln(0.90/0.05) = \ln(18) \approx 2.89$
- $B = \ln(0.10/0.95) \approx -2.25$

Probability thresholds. Inverting the logit function yields interpretable probability cutoffs:

$$p^{\text{unsafe}} = \frac{1-\beta}{1-\beta+\alpha} = \frac{0.90}{0.95} \approx 0.947, \quad p^{\text{safe}} = \frac{\beta}{1-\alpha+\beta} = \frac{0.10}{1.05} \approx 0.095 \tag{10}$$

5.3 Prior Adjustment for Imbalanced Distributions

In real-world deployments, the base toxicity rate $\pi = P(H_1)$ is typically low ($< 5\%$). When the classifier was trained on balanced data but deployed on imbalanced data, we can adjust thresholds to account for the prior:

$$A' = A + \ln \frac{\pi}{1-\pi}, \quad B' = B + \ln \frac{\pi}{1-\pi} \tag{11}$$

Algorithm 1 Cascade-Guard: Two-Stage Streaming Safety Detection

Input: Token stream, Stage 1 classifier f^{S1}, Stage 2 classifier g^{S2}, Parameters (α, β, π), Min tokens $t_{\min}$, Max tokens $T_{\max}$
Compute thresholds: $A' = \ln \frac{1-\beta}{\alpha} + \ln \frac{\pi}{1-\pi}$, $B' = \ln \frac{\beta}{1-\alpha} + \ln \frac{\pi}{1-\pi}$
for each token x_t from $t = t_{\min}$ **do**
  Stage 1: $p_t = f_\theta^{S1}(x_1, \ldots, x_t)$
  if $p_t \geq \sigma(A')$ (Stage 1 flags UNSAFE) **then**
    Stage 2: result $= g_\phi^{S2}(x_1, \ldots, x_t)$
    if result is UNSAFE **then**
      return UNSAFE (both stages agree)
    end if{Stage 2 rejects: false alarm, continue generation}
  end if
  if $p_t \leq \sigma(B')$ **then**
    return SAFE
  end if
end for

return SAFE (reached $T_{\max}$)

Intuition. When toxicity is rare ($\pi \ll 0.5$), $\ln \frac{\pi}{1-\pi} < 0$, shifting *both* thresholds downward. This makes it easier to declare UNSAFE and harder to declare SAFE, which compensates for the low base rate.

Remark 5.2 (Critical Limitation)**.** This prior adjustment invalidates the Wald error bounds. The classical proof requires thresholds derived solely from (α, β). Shifting by $\ln \frac{\pi}{1-\pi}$ introduces a bias that can cause empirical FPR to far exceed α. We empirically characterize this in Section 7.2.6.

5.4 Algorithm Summary

5.5 Relationship to Confidence-Based Early Exit

The single-stage Wald method is mathematically equivalent to confidence-based early exit: the decision $\log \frac{p_t}{1-p_t} \geq A'$ is identical to $p_t \geq \sigma(A')$. For example, Wald(α=0.05, β=0.10, π=0.01) produces $A' = -1.70$, which corresponds to the confidence threshold $\tau = \sigma(-1.70) \approx 0.154$.

This means that *any* operating point achievable by Wald is also achievable by some confidence threshold $\tau \in [0, 1]$. The Wald parameter space provides a semantically meaningful way to navigate the Pareto frontier, but does not produce new points on it.

The (α, β, π) parameterization offers three concrete advantages over direct threshold search:

(1) **Monotonic control**: Decreasing π consistently lowers the UNSAFE threshold and increases recall (Section 7.2.8), providing predictable behavior that grid search over τ does not guarantee.
(2) **Interpretability**: Practitioners set α to their maximum tolerable FPR, β to their maximum tolerable FNR, and π to their estimated toxicity rate—all directly encoding deployment requirements rather than requiring trial-and-error threshold tuning.
(3) **Composability with cascade**: The prior-adjusted threshold for Stage 1 is set independently from Stage 2's decision boundary, enabling joint optimization without coupling two models' hyperparameters.

This motivates the heterogeneous two-stage cascade: by requiring agreement between two fundamentally different models, the cascade produces operating points that no single model's threshold sweep can achieve.

5.6 Cascade-Guard: Heterogeneous Two-Stage Architecture

The fundamental limitation of any single-threshold method is the precision-recall trade-off: lowering the threshold improves recall (fewer missed toxic content) but always increases FPR. In safety-critical applications, we need both high recall and controlled FPR—an impossibility for any single classifier operating at a single threshold.

The Cascade-Guard architecture attempts to resolve this tension through model diversity.

Stage 1 (RoBERTa Screening): A fine-tuned RoBERTa-base (125M params, ~7ms/call) monitors each prefix during streaming generation. Using Wald-derived thresholds with aggressive priors ($\pi \ll 0.5$), Stage 1 flags potentially unsafe content with near-zero miss rate but high FPR.

Stage 2 (Llama Guard Confirmation): When Stage 1 flags a prefix, Llama Guard (8B params, ~86ms/call) classifies the same prefix. Only if both models agree on UNSAFE does the cascade declare the content unsafe. Otherwise, generation continues.

Decision rule.

$$\delta_{\text{cascade}}(t) = \begin{cases} \text{UNSAFE} & \text{if } f_\theta^{S1}(x_{\leq t}) \geq \sigma(A') \textbf{ and } g_\phi^{S2}(x_{\leq t}) = \text{unsafe} \\ \text{SAFE} & \text{if } f_\theta^{S1}(x_{\leq t}) \leq \sigma(B') \\ \text{CONTINUE} & \text{otherwise} \end{cases} \tag{12}$$

where f_θ^{S1} and g_ϕ^{S2} are the Stage 1 (RoBERTa) and Stage 2 (Llama Guard) classifiers respectively.

Why is this not equivalent to a single threshold? The two stages use fundamentally different models—RoBERTa is a 125M-parameter encoder trained on toxicity labels, while Llama Guard is an 8B-parameter generative model trained for safety judgment. Their outputs $f_\theta^{S1}(x)$ and $g_\phi^{S2}(x)$ are neither identical nor deterministically related. For a given input x, the condition $(p_x^{S1} \geq \tau_1) \wedge (\text{unsafe}_x^{S2})$ cannot be expressed as $p_x^{S1} \geq \tau^*$ for any single threshold τ^*, because g_ϕ^{S2} is an independent model with its own decision boundary. This is analogous to ensemble methods: the intersection of two independent classifiers' positive regions produces a decision boundary that neither classifier alone can achieve.

Stage 2 operates on the same prefix where Stage 1 triggered, requiring no additional token generation. The overhead is one Llama Guard forward pass at the stopping point, costing ~86ms. The token savings are identical to the Stage 1 screening, since generation stops at the same point.

The intuition for FPR reduction is straightforward: the RoBERTa screening catches nearly all toxic content (low FNR) but also flags many safe samples (high FPR). Llama Guard acts as a second opinion from a fundamentally different model—it catches the false alarms because its errors are uncorrelated with RoBERTa's. The probability that both models make a false positive error on the same input is much lower than either model alone.

6 Theoretical Analysis

We present the classical SPRT theory for completeness, then clearly identify where our method deviates.

6.1 Classical SPRT Error Bounds (Under Ideal Conditions)

Theorem 6.1 (Wald's Error Bounds—Classical). *Under i.i.d. observations with S_t^{true} equaling the true log-likelihood ratio, Wald's SPRT satisfies:*

$$\mathbb{P}^{H_0}(FP) \leq \frac{\alpha}{1-\beta} \leq \alpha, \quad \mathbb{P}^{H_1}(FN) \leq \frac{\beta}{1-\alpha} \leq \beta \tag{13}$$

The proof uses the martingale property of $\exp(S_t^{\text{true}})$ under H_0 and the optional stopping theorem. This is a standard result [1].

6.2 Why Theoretical Guarantees Do Not Hold in Our Setting

Our practical implementation violates all assumptions required for Theorem 6.1:

1. **Classifier approximation**: We use $\tilde{S}_t = \log \frac{p_t}{1-p_t}$ instead of the true log-likelihood ratio S_t^{true}. Neural classifiers exhibit miscalibration (ECE typically 3–8%), causing $\tilde{S}_t \neq S_t^{\text{true}}$.

2. **Non-i.i.d. observations**: LLM tokens are highly dependent. The martingale property required for Wald's proof depends on the correctness of the underlying probabilistic model.

3. **Prior adjustment**: When we apply Equation 11, we shift both thresholds by $\ln \frac{\pi}{1-\pi}$. For $\pi = 0.01$:

$$\ln \frac{0.01}{0.99} \approx -4.60 \tag{14}$$

This shifts A from 2.89 to -1.70, making it much easier to declare UNSAFE, which dramatically increases empirical FPR.

However, this prior adjustment is precisely what makes the framework effective in practice: without it, the threshold $A \approx 2.89$ is too high for most classifiers to ever cross. The trade-off is that we sacrifice theoretical guarantees for practical utility. The heterogeneous two-stage cascade (Section 5.6) mitigates this trade-off by using Llama Guard as Stage 2 to control the high FPR that results from aggressive prior adjustment.

While full error rate guarantees do not hold, Wald's formula retains useful properties:

(1) **Monotonic relationship**: Increasing α consistently lowers the UNSAFE threshold, increasing empirical FPR. The directional relationship between parameters and error rates is preserved.
(2) **Interpretable parametrization**: (α, β) directly encode the relative strictness of SAFE vs. UNSAFE decisions, providing a clear interface for practitioners.
(3) **Principled initialization**: Wald thresholds provide a theoretically motivated starting point, reducing the search space for threshold tuning compared to blind grid search.

An important note on the cascade's FPR reduction: the cascade's empirical FPR reduction comes from model diversity (RoBERTa and Llama Guard have independent error modes), not from SPRT theory. The SPRT derivation provides the Stage 1 screening threshold; the FPR improvement is an architectural consequence of using two different models.

7 Experiments

7.1 Experimental Setup

7.1.1 Datasets. We use four diverse datasets spanning low-toxicity, high-toxicity, and adversarial scenarios:

Note that Qwen3GuardTest contains only toxic samples (100% toxicity rate). FPR is therefore trivially 0 for all methods on this dataset, since there are no safe samples to falsely flag. We include this dataset primarily to evaluate recall and token savings under adversarial conditions.

Table 1. Dataset Characteristics

Dataset	Test Size	Toxicity Rate	Content Type
CivilComments	5,000	8.0%	User comments
BeaverTails	907	57.3%	LLM responses
PKU-SafeRLHF	3,000	52.0%	Preference data
Qwen3GuardTest	489	100%	Adversarial

7.1.2 Stage 1: RoBERTa Classifier. We use RoBERTa-base [11] (125M parameters) fine-tuned on the training split of each respective dataset. Each classifier is trained with binary cross-entropy loss for 3 epochs with learning rate 2×10^{-5}. We apply temperature scaling [2] for calibration, reducing ECE from 4.4–7.6% (pre-calibration) to 2.0–5.8%.

7.1.3 Stage 2: Llama Guard. We use Llama Guard 3 8B [9, 15] as the Stage 2 classifier. Llama Guard is applied without fine-tuning across all datasets—it provides zero-shot safety classification. This is a key advantage: the Stage 2 model generalizes across domains without dataset-specific training.

7.1.4 Comparison Methods.

- **Full-text RoBERTa**: Post-hoc classification on complete text using RoBERTa (upper bound for Stage 1)
- **Full-text Llama Guard**: Post-hoc classification on complete text using Llama Guard (upper bound for Stage 2, and precision upper bound)
- **Wald streaming(π)**: Single-stage RoBERTa with Wald-derived thresholds and prior π ($\alpha = 0.05, \beta = 0.10$)
- **Confidence(τ)**: Single-stage RoBERTa with confidence-based early exit, full sweep $\tau \in \{0.10, 0.15, 0.20, 0.30, 0.50, 0.70, 0.80, 0.90, 0.95\}$
- **Cascade-Guard(π)**: Our heterogeneous two-stage cascade (RoBERTa + Llama Guard) with Wald screening prior π
- **Fixed-Wk**: Fixed-window detection every k tokens

7.1.5 Implementation Details.

- Minimum tokens: $t_{\min} = 5$, Maximum tokens: $T_{\max} = 128$
- Default error rates: $\alpha = 0.05$, $\beta = 0.10$, thresholds $A \approx 2.89$, $B \approx -2.25$
- Stage 1 (RoBERTa): ~7ms per call on NVIDIA RTX 4090
- Stage 2 (Llama Guard 3 8B): ~86ms per call on NVIDIA RTX 4090
- GPU: NVIDIA GeForce RTX 4090

7.2 Main Results

7.2.1 Overall Performance. Several patterns emerge from these results.

First, Stage 2 overrules hurt cascade performance. On PKU-SafeRLHF, single-stage Wald(π=0.01) achieves F1=0.513 with FPR=10.1%. The cascade adds Llama Guard, which further reduces FPR to 5.1% but at the cost of F1=0.416 because Llama Guard overrules many correct Stage 1 decisions on short prefixes. On Qwen3GuardTest, cascade F1=0.238 is catastrophically low because Llama Guard has 46.2% FNR even on full text, and overrules 86.5% of correct Stage 1 unsafe decisions.

Second, all single-stage methods succeed on adversarial data. On Qwen3GuardTest (100% toxic), fine-tuned RoBERTa achieves perfect detection (F1=1.0) regardless of threshold—both Wald-parameterized and confidence methods detect

Table 2. Main Results: Cascade-Guard vs. Single-Stage Methods across Four Datasets. Best F1 per dataset block in **bold**. "FT-RoBERTa" and "FT-LG" denote full-text baselines. "LG-Calls" shows number of Llama Guard invocations.

Dataset	Method	Acc	F1	FPR	FNR	AvgTok	Savings	LG-Calls
Civil Comments (8% tox.)	FT-RoBERTa	0.941	0.637	0.033	0.354	128	0%	0
	FT-LG	0.913	0.138	0.016	0.912	128	0%	5000
	Cascade(π=0.01)	0.920	0.052	0.002	0.972	9.7	92.5%	338
	Wald(π=0.01)	0.905	**0.356**	0.045	0.671	9.7	92.5%	0
	Conf(0.95)	0.929	0.522	0.033	0.513	49.9	61.0%	0
Beaver Tails (57% tox.)	FT-RoBERTa	0.803	0.816	0.145	0.237	128	0%	0
	FT-LG	0.686	0.682	0.181	0.413	128	0%	907
	Cascade(π=0.1)	0.588	0.530	0.168	0.594	37.0	71.1%	577
	Wald(π=0.1)	0.734	**0.780**	0.385	0.177	37.0	71.1%	0
	Conf(0.5)	0.682	0.769	0.641	0.077	22.9	82.1%	0
PKU-SafeRLHF (52% tox.)	FT-RoBERTa	0.930	0.933	0.069	0.070	128	0%	0
	FT-LG	0.867	0.875	0.162	0.106	128	0%	3000
	Cascade(π=0.01)	0.598	0.416	0.051	0.725	8.0	93.8%	734
	Wald(π=0.01)	0.627	0.513	0.101	0.623	8.0	93.8%	0
	Conf(0.5)	0.842	0.866	0.303	0.024	47.8	62.7%	0
	Fixed-W10	0.868	0.884	0.233	0.038	53.3	58.3%	0
Qwen3 Guard (100% tox.)	FT-RoBERTa	1.000	1.000	0	0.0	128	0%	0
	FT-LG	0.538	0.699	0.000	0.462	128	0%	489
	Cascade(π=0.05)	0.135	0.238	0.000	0.865	5.0	96.1%	489
	Wald(π=0.05)	1.000	1.000	0	0.0	5.0	96.1%	0
	Conf(0.95)	1.000	1.000	0	0.0	5.0	96.1%	0
	Fixed-W5	1.000	1.000	0	0.0	5.0	96.1%	0

all unsafe content with 96.1% token savings. The cascade is the only method that fails (F1=0.238), because Llama Guard overrules 86.5% of correct Stage 1 unsafe decisions. Note that because Qwen3GuardTest contains no safe samples, FPR is trivially 0 for all methods and F1 equals Recall; the meaningful metric for this dataset is Recall (i.e., what fraction of adversarial content is detected).

Third, the cascade preserves token savings. Since Stage 2 (Llama Guard) operates on the same prefix where Stage 1 (RoBERTa) triggered, the token savings are identical to the single-stage method. The overhead is one Llama Guard forward pass (∼86ms) only for flagged samples.

Finally, the number of Llama Guard calls serves as an efficiency metric. Full-text LG requires calling Llama Guard on every sample. Cascade-Guard only invokes Llama Guard when Stage 1 flags a prefix. On PKU-SafeRLHF with π=0.01, this is 734/3000 (24.5%) samples; on Qwen3GuardTest with π=0.05, Stage 1 triggers on all 489 test samples (100%), requiring LG for every sample.

7.2.2 Cascade FPR Reduction Analysis. On BeaverTails, the cascade achieves 56% FPR reduction (0.385→0.168) but F1 drops from 0.780 to 0.530. On PKU-SafeRLHF, it achieves 50% FPR reduction (0.101→0.051) but F1 drops from 0.513 to 0.416. On CivilComments, FPR reduction is 96% (0.045→0.002) but F1 collapses to 0.052. On Qwen3GuardTest, all single-stage methods achieve perfect detection (F1=1.0); the cascade is the only failure (F1=0.238) because Llama Guard overrules 86.5% of correct Stage 1 unsafe decisions. The pattern is clear: cascade effectiveness is bounded by Stage 2's base performance.

Table 3. FPR Reduction: Single-Stage Wald vs. Cascade-Guard. Best configuration per dataset.

Dataset	Method	F1	FPR	FNR	Savings	LG-Calls
BeaverTails	Wald(π=0.1)	0.780	0.385	0.177	71.1%	0
	Cascade(π=0.1)	0.530	0.168	0.594	71.1%	577
PKU-SafeRLHF	Wald(π=0.01)	0.513	0.101	0.623	93.8%	0
	Cascade(π=0.01)	0.416	0.051	0.725	93.8%	734
CivilComments	Wald(π=0.01)	0.356	0.045	0.671	92.5%	0
	Cascade(π=0.01)	0.052	0.002	0.972	92.5%	338

7.2.3 Prefix Failure Analysis: Why Llama Guard Overrules Correct Decisions. To understand why the cascade underperforms, we extract cases where RoBERTa (Stage 1) correctly flagged a prefix as UNSAFE but Llama Guard (Stage 2) overruled it as SAFE—the ground truth is UNSAFE. We found 799 such failures across all four datasets. Table 4 shows representative examples.

Table 4. Prefix Failure Cases: RoBERTa correctly flags UNSAFE but Llama Guard overrules as SAFE. Truncated for readability.

#	Prefix Text (truncated)	S1 Prob	Tokens	Dataset
1	"What are some insults"	0.988	5	PKU
2	"How do I hide"	0.9996	5	Qwen3
3	"So I've heard" (garrote wire)	0.9996	5	Qwen3
4	"Can you find me" (incel groups)	0.9997	5	Qwen3
5	"How can I make someone feel bad"	0.994	8	PKU
6	"…bioterrorism…dispersed through"	0.200	72	PKU

We observe two distinct failure modes. First, short-prefix blindness (Examples 1–5): when Stage 1 stops at $t_{\min}$=5 tokens with high confidence (prob > 0.98), the prefix is often ambiguous in isolation. "How do I hide" could be followed by "food from my parents" (benign) or "a body" (dangerous). Llama Guard, operating on this short prefix, defaults to SAFE. Second, long-prefix failures (Example 6): even with 72 tokens of context containing explicit bioterrorism language, Llama Guard classifies the prefix as SAFE, suggesting that its safety training does not adequately cover certain harm categories (e.g., CBRNE threats) in streaming contexts.

More broadly, Llama Guard exhibits a conservative bias: as a zero-shot classifier applied without fine-tuning, it tends toward SAFE predictions on short prefixes, producing 46.2% FNR on Qwen3GuardTest even on full text. This systematic bias causes the cascade to lose recall.

These failures reveal what we call the prefix bottleneck: Llama Guard was trained on complete texts, not streaming prefixes. When applied to short prefixes, its zero-shot classification defaults to SAFE, overruling correct Stage 1 decisions. Fine-tuning Llama Guard on short prefixes could mitigate this issue—we evaluate this in Section 7.2.4.

7.2.4 Prefix-Aware Stage 2: Impact of Fine-Tuning Llama Guard. To address the prefix bottleneck identified above, we fine-tune Llama Guard 3 8B on prefix data using QLoRA (4-bit quantization, LoRA rank 16, LoRA alpha 32, targeting

attention modules $q_proj, k_proj, v_proj, o_proj$). Training data consists of 40K prefix samples (20K safe + 20K unsafe) extracted at multiple token positions (5, 10, 15, 20, 30, 50, 72) from the training splits of BeaverTails, PKU-SafeRLHF, and Qwen3GuardTest, labeled with the ground-truth safety of the full text. Training completed 2,000/2,250 steps (89% of 1 epoch) on RTX 4090.

Table 5. Prefix-Aware Stage 2: Zero-Shot vs. Fine-Tuned Llama Guard in the Cascade. QLoRA adapter trained on 40K prefix samples. Best F1 per dataset in **bold**.

Dataset	Method	F1	Prec	Rec	FPR	FNR	Prefix Failures
BeaverTails	Cascade-ZS	0.559	0.766	0.440	0.181	0.560	187
	Cascade-FT	**0.723**	**0.834**	**0.638**	0.171	**0.362**	**84**
PKU-SafeRLHF	Cascade-ZS	0.256	0.842	0.151	0.031	0.849	89
	Cascade-FT	**0.319**	**0.871**	**0.196**	0.031	**0.804**	**19**
Qwen3GuardTest	Cascade-ZS	0.238	1.000	0.135	0.000	0.865	423
	Cascade-FT	**0.736**	1.000	**0.583**	0.000	**0.417**	**204**

Prefix-aware fine-tuning substantially improves cascade F1 across all three datasets. On BeaverTails, Cascade-FT achieves F1=0.723 vs. Cascade-ZS F1=0.559 (+29.4%). On Qwen3GuardTest, F1 jumps from 0.238 to 0.736 (+209%), demonstrating that Llama Guard's prefix-level judgment can be improved through targeted training. Correspondingly, prefix failures are reduced by 52–79%: from 89 to 19 on PKU-SafeRLHF (78.7%), from 187 to 84 on BeaverTails (55.1%), and from 423 to 204 on Qwen3GuardTest (51.8%). Notably, Cascade-FT maintains low FPR on all datasets (0.171 on BeaverTails, 0.031 on PKU-SafeRLHF, 0.000 on Qwen3GuardTest), so fine-tuning improves recall without sacrificing precision.

Despite the largest relative improvement, Qwen3GuardTest remains challenging. Cascade-FT achieves F1=0.736, compared to single-stage Wald streaming's perfect F1=1.000. The 204 remaining prefix failures suggest that some adversarial patterns are too ambiguous for Llama Guard to detect from short prefixes, even after fine-tuning.

Note that the Cascade-ZS baselines in this table were re-run as part of the same experimental pipeline (with independently trained RoBERTa models) and may differ slightly from the main results in Table 2. Comparing with single-stage Wald streaming: on BeaverTails, Cascade-FT (F1=0.723) approaches single-stage Wald streaming (F1=0.780) much more closely than Cascade-ZS (F1=0.559), reducing the cascade F1 gap from 28.3% to 7.3%. This validates the cascade architecture's potential: with a prefix-aware Stage 2, the cascade can provide FPR reduction (0.171 vs. 0.385 for single-stage Wald streaming) with only moderate F1 cost.

7.2.5 Full-Text Performance of Prefix-Tuned Llama Guard. A natural concern is whether prefix-aware fine-tuning degrades Llama Guard's performance on complete texts. To address this, we evaluate both zero-shot and prefix-fine-tuned Llama Guard 3 8B on full-text inputs from BeaverTails and PKU-SafeRLHF test sets. Both models use 4-bit NF4 quantization; the fine-tuned model additionally loads the QLoRA adapter from Section 7.2.4 with merge-and-unload.

Prefix-aware fine-tuning does *not* degrade full-text performance—it improves it across all metrics. On BeaverTails, F1 increases from 0.687 to 0.855 (+24.4%), driven by both higher precision (0.852→0.951) and higher recall (0.575→0.777). FPR improves from 13.4% to 5.4%. On PKU-SafeRLHF, F1 increases from 0.881 to 0.911 (+3.4%), with FPR improving from 14.4% to 11.2%.

This improvement is consistent with transfer learning theory: training on diverse prefix lengths (5–72 tokens) exposes the model to a broader distribution of input contexts than full-text-only training. The prefix samples act as

Table 6. Full-Text Evaluation: Zero-Shot vs. Prefix-Fine-Tuned Llama Guard 3 8B on complete texts. Both models use 4-bit NF4 quantization. Best per dataset in **bold**.

Dataset	Version	Prec	Rec	F1	FPR	FNR
BeaverTails	Zero-Shot	0.852	0.575	0.687	0.134	0.425
	Fine-Tuned	**0.951**	**0.777**	**0.855**	0.054	**0.223**
PKU-SafeRLHF	Zero-Shot	0.873	0.890	0.881	0.144	0.110
	Fine-Tuned	**0.901**	**0.920**	**0.911**	0.112	**0.080**

a form of data augmentation, and the QLoRA adapter's low rank (r=16) constrains the update to a low-dimensional subspace that does not overwrite full-text knowledge. These results confirm that prefix-aware fine-tuning is a safe upgrade: it improves both cascade performance on short prefixes (Table 5) and standalone full-text classification.

7.2.6 Wald Bounds Validation. A critical question is whether the theoretical error rate parameters (α, β) bear any relationship to empirical error rates:

Table 7. Wald Bounds Validation: Empirical FPR vs. Target $\alpha = 0.05$ across All Datasets and Priors. Deviation = Empirical FPR/α. π^* = dataset toxicity rate.

Dataset	π^*	Prior π	Emp. FPR	Dev.	Holds?
CivilComments (8%)	0.080	0.50	0.008	0.16×	Yes
	0.080	0.10	0.014	0.29×	Yes
	0.080	0.05	0.016	0.32×	Yes
	0.080	0.01	0.038	0.75×	Yes
BeaverTails (57%)	0.574	0.50	0.156	3.1×	No
	0.574	0.10	0.459	9.2×	No
	0.574	0.05	0.599	12.0×	No
	0.574	0.01	0.929	18.6×	No
PKU-SafeRLHF (52%)	0.520	0.50	0.162	3.2×	No
	0.520	0.10	0.299	6.0×	No
	0.520	0.05	0.382	7.6×	No
	0.520	0.01	0.586	11.7×	No
Qwen3Guard (100%)	1.000	0.50	0.000	—	Trivial
	1.000	0.10	0.000	—	Trivial
	1.000	0.05	0.000	—	Trivial
	1.000	0.01	0.000	—	Trivial

The expanded analysis across 16 configurations reveals three patterns. First, bounds hold only when $\pi \approx \pi^*$: Wald bounds hold (FPR $\leq \alpha$) on CivilComments where $\pi^* = 8\%$, and even $\pi = 0.01$ produces FPR = 0.038 < 0.05. But on balanced datasets ($\pi^* \approx 52$–57%), every prior setting violates the bound because the classifier's raw probability distribution is already centered around high values for toxic-majority data.

Second, deviation grows monotonically with $|\log(\pi/\pi^*)|$. On BeaverTails, deviation ranges from 3.1× ($\pi = 0.5$) to 18.6× ($\pi = 0.01$). The prior adjustment $\ln(\pi/(1-\pi))$ shifts thresholds by up to -4.60 (for $\pi = 0.01$), which overwhelms the Wald threshold $A = 2.89$ and produces extremely permissive stopping.

Third, as a practical guideline, set π to the estimated toxicity rate $\hat{\pi}$. The deviation remains $\leq 1\times$ when $\pi \geq \pi^*/5$. For environments where π^* is unknown, start with $\pi = 0.10$ and adjust downward based on observed FPR.

These results confirm that Wald thresholds provide useful screening thresholds with predictable monotonic behavior, but not guaranteed error bounds—their tightness depends on the gap between π and π^*. The cascade architecture mitigates FPR violations by adding a confirmation stage.

7.2.7 Pareto Frontier: Cascade vs. Single-Threshold Methods. We compare the best Cascade-Guard configuration against the best single-threshold methods (both Wald-parameterized and confidence).

Table 8. Pareto-Optimal Operating Points: Best Cascade-Guard vs. Best Single-Threshold Methods

Dataset	Method	Savings	F1	FNR	FPR	Winner
BeaverTails	Cascade(π=0.1)	71.1%	0.530	59.4%	16.8%	Wald
	Wald(π=0.1)	71.1%	**0.780**	17.7%	38.5%	
	Conf(0.5)	82.1%	0.769	7.7%	64.1%	
PKU-SafeRLHF	Cascade(π=0.01)	93.8%	0.416	72.5%	5.1%	Wald
	Wald(π=0.01)	93.8%	**0.513**	62.3%	10.1%	
	Conf(0.5)	62.7%	0.866	2.4%	30.3%	
Qwen3Guard	Cascade(π=0.05)	96.1%	0.238	86.5%	0.0%	Tie
	Wald(π=0.05)	96.1%	**1.000**	0.0%	0.0%	
	Conf(0.95)	96.1%	**1.000**	0.0%	0.0%	Tie

On BeaverTails, single-stage Wald(π=0.1) achieves the highest F1 (0.780) at the same savings level (71.1%) as Cascade(π=0.1), while Cascade provides a substantially lower FPR (16.8% vs. 38.5%). On PKU-SafeRLHF, Conf(0.5) achieves the highest F1 (0.866) with 62.7% savings, while Wald(π=0.01) achieves 93.8% savings but at F1=0.513. On Qwen3GuardTest, all single-stage methods (Wald-parameterized, confidence, fixed-window) achieve perfect detection (F1=1.0) with 96.1% savings—fine-tuned RoBERTa easily detects adversarial patterns. The cascade is the only method that fails (F1=0.238) because Llama Guard overrules 86.5% of correct Stage 1 unsafe decisions.

Table 9. Prior Sensitivity across Datasets ($\alpha = 0.05, \beta = 0.10$)

Dataset	Prior π	FPR	FNR	F1	Savings
BeaverTails	0.50	0.137	0.408	0.699	63.5%
	0.10	0.385	0.177	**0.780**	71.1%
	0.05	0.680	0.069	0.764	83.4%
	0.01	1.000	0.000	0.729	96.1%
PKU-SafeRLHF	0.50	0.009	0.933	0.124	96.1%
	0.10	0.013	0.929	0.131	96.1%
	0.05	0.017	0.923	0.141	96.0%
	0.01	0.101	0.623	**0.513**	93.8%
Qwen3Guard	0.10	0.000	0.049	0.975	79.2%
	0.05	0.000	0.000	**1.000**	96.1%
	0.01	0.000	0.000	**1.000**	96.1%

7.2.8 Prior Sensitivity Analysis. The optimal π depends on dataset toxicity: $\pi \approx 0.10$ for BeaverTails (57%), $\pi \approx 0.01$ for PKU-SafeRLHF (52%), $\pi \leq 0.05$ for Qwen3GuardTest (100%). Lower π consistently reduces FNR but increases FPR; the FNR reduction is dramatic: from 40.8% to 0.0% on BeaverTails as π goes from 0.5 to 0.01. As a practical guideline, set π

to the approximate toxicity rate of the deployment environment. For unknown environments, start with $\pi = 0.05$ and adjust based on observed FPR/FNR.

7.2.9 Late-Toxicity Detection (Qwen3GuardTest). Qwen3GuardTest provides position annotations for toxic content, enabling analysis of **late-toxicity**: adversarial content where toxicity appears only after a benign prefix.

Table 10. Late-Toxicity Detection on Qwen3GuardTest ($\pi = 0.05$, $\alpha = 0.05$, $\beta = 0.10$). Analysis on full 651-sample dataset (489 test, 162 train).

Category	Count	Detection Rate	Avg Stop Token
Early-toxic	253	100.0%	5.0
Late-toxic	398	100.0%	5.0

Wald(π=0.05) achieves 100% detection rate on both early-toxic and late-toxic adversarial samples, with average stopping at just 5 tokens (the minimum). The prior-adjusted threshold $A' = 2.89 + \ln(0.05/0.95) \approx -0.59$ catches toxic content extremely early, even when toxicity appears late in the sequence.

With the default prior (π=0.5), the detector still achieves FNR=0% and F1=1.0—the fine-tuned RoBERTa detects adversarial content at all prior settings. The key insight is that once RoBERTa has been fine-tuned on adversarial patterns, even high Wald thresholds are crossed early in generation.

Table 11. Calibration Performance across Datasets (Fine-tuned RoBERTa-base)

Dataset	ECE (Raw)	ECE (Temp Scaling)	ECE (Isotonic)
CivilComments	4.4%	3.9%	≈0%
BeaverTails	7.0%	5.8%	≈0%
PKU-SafeRLHF	4.6%	3.7%	≈0%
Qwen3GuardTest	7.6%	2.0%	≈0%

7.2.10 Calibration Analysis. Fine-tuned classifiers achieve ECE of 4–8% before calibration, significantly better than the pre-trained classifier's 21% ECE on CivilComments. Temperature scaling reduces ECE to 2–6%, and isotonic regression achieves near-perfect calibration (<0.01% ECE).

Table 12. Impact of Calibration on Wald Performance ($\pi = 0.01$, $\alpha = 0.05$, $\beta = 0.10$)

Dataset	Calibration	F1	FPR	FNR
BeaverTails	Raw	0.741	0.929	0.005
	Temp Scaled	0.729	1.000	0.000
PKU-SafeRLHF	Raw	0.797	0.317	0.144
	Temp Scaled	0.829	0.424	0.014

7.2.11 Effect of Calibration on Wald Thresholds. Temperature scaling has dataset-dependent effects. On BeaverTails, it slightly decreases F1 (0.741→0.729) because FPR was already near ceiling (0.929→1.000). On PKU-SafeRLHF, temperature scaling improves F1 by 4% (0.797→0.829) through a dramatic FNR reduction (14.4%→1.4%) at the cost of higher FPR

(31.7%→42.4%). The calibrated model (temperature T=1.160) pushes probability mass away from the decision boundary, making the classifier more decisive—this helps when the raw model is underconfident on true positives.

7.2.12 Computational Overhead Analysis. We present both analytical and empirically validated latency measurements. The end-to-end latency has three components: LLM generation, Stage 1 RoBERTa classification, and Stage 2 Llama Guard confirmation.

Empirical validation. We integrate our pipeline with actual LLM text generation using Llama Guard 3 8B (FP16) as an 8B-parameter causal language model on NVIDIA RTX 4090. Table 13 compares measured component latencies with our analytical estimates.

Table 13. Component Latency: Measured vs. Analytical Estimates (RTX 4090, FP16)

Component	Measured	Analytical	Ratio
LLM generation (ms/tok)	27.2±4.6	50.0 (est.)	0.54×
RoBERTa S1 (ms/call)	7.0±0.4	6.1 (meas.)	1.15×
Llama Guard S2 (ms/call)	86.2	∼100 (meas.)	0.86×

The analytical model is **conservative**: all measured latencies are at or below the estimates. The LLM generation estimate of 50ms/tok was chosen to represent larger models (GPT-4 class); the measured 27.2ms/tok for an 8B model on RTX 4090 confirms this is a reasonable upper bound.

Table 14 shows the end-to-end pipeline latencies computed from measured values.

Table 14. End-to-End Pipeline Latency (Measured on RTX 4090, FP16). r = Stage 2 trigger rate.

Method	LLM Gen	S1	S2	Speedup
Full-text (128 tok)	3,484ms	7ms	–	1.0×
Llama Guard only	3,484ms	–	86ms	1.0×
Wald(π=0.01)	435ms	88ms	–	7.0×
Cascade(π=0.01)	435ms	88ms	86ms	6.5×

where the Cascade speedup assumes Stage 2 is invoked on $r = 100\%$ of flagged samples at the stopping point. With trigger rate $r < 1$, the amortized speedup is higher. On PKU-SafeRLHF with π=0.01 and $r \approx 0.59$, the measured total is ∼570ms vs. 3,570ms for full-text Llama Guard—a **6.3× speedup** with cascade confirmation.

Key advantage: Llama Guard is invoked only on the $\sim r$ fraction of flagged samples, not on every token or every sample. For most deployment scenarios ($r < 0.3$), the amortized Stage 2 overhead is $r \times 86 \approx 26$ms per generation—negligible compared to the ∼3.5s saved by early stopping.

7.2.13 Latency Jitter Analysis. A practical concern is the impact of Stage 2 invocation on user-perceived latency. In a streaming deployment with our tested 8B model, each token arrives at ∼27ms intervals (measured on RTX 4090). Our default architecture uses **synchronous execution**: when Stage 1 flags a prefix, LLM generation pauses while Stage 2 (Llama Guard) evaluates the prefix. This introduces a single mid-generation pause of ∼86ms (measured). The alternative is **speculative execution**: continue generating tokens during Stage 2 inference and discard buffered tokens if Stage 2 confirms UNSAFE. Speculative execution eliminates the pause but wastes compute on tokens that may be discarded. In

our experiments, the synchronous mode is used, with the pause occurring at most once per generation at the stopping point.

Table 15 shows the Stage 1 trigger rates and **measured** Time-Between-Tokens (TBT) across datasets. The normal TBT is 34.2ms (LLM 27.2ms + RoBERTa 7.0ms); when Stage 2 triggers, the single pause adds 86ms, giving a peak TBT of 120.3ms.

Table 15. Stage 1 Trigger Rates and Measured Latency Impact. r = fraction of samples triggering Stage 2. TBT values are **measured** on RTX 4090 with an 8B-parameter LLM (FP16).

Dataset	Prior π	Trigger r	Peak TBT	P95 TBT	Samples Affected
BeaverTails	0.10	63.6%	120ms	120ms	577/907
	0.50	15.6%	120ms	35ms	141/907
PKU-SafeRLHF	0.01	24.5%	120ms	50ms	734/3000
	0.10	1.7%	120ms	35ms	51/3000
CivilComments	0.01	6.8%	120ms	35ms	338/5000
Qwen3GuardTest	0.05	100%	120ms	120ms	489/489

For most deployment scenarios, the Stage 1 trigger rate is moderate (1.7–24.5%), meaning only a minority of samples experience the ~86ms pause. The 95th percentile Time-Between-Tokens (P95 TBT) remains below 50ms for balanced datasets with practical priors ($\pi \geq 0.05$), well within acceptable thresholds for streaming applications (typically <200ms).

The worst case occurs when Stage 1 trigger rate is very high: on Qwen3GuardTest with π=0.05, every sample triggers Stage 2, producing a single 86ms pause per generation (peak TBT=120ms). This is still far less than the ~3.5s full-generation latency avoided by early stopping. In deployment, if jitter is a concern, the speculative execution mode described above can eliminate the pause entirely at the cost of potentially wasted computation.

Chunking strategy: Check every k tokens instead of every token:

Table 16. Chunking Trade-offs (CivilComments, $\pi = 0.01$, $\alpha = 0.05$, $\beta = 0.10$)

k	F1	FPR	Avg Calls	Token Savings
1 (every token)	0.183	81.6%	12.6	87.0%
3	0.189	78.8%	5.6	85.2%
5	0.188	77.0%	3.8	84.8%
10	0.190	74.2%	2.5	83.3%

Chunking provides substantial reduction in classifier calls (from 12.6 to 2.5) with minimal F1 impact (0.183 vs. 0.190). Larger chunk sizes slightly reduce FPR because the classifier sees more context before making a decision. The FNR increases modestly with chunk size (2.3% at k=1 to 7.0% at k=10) because toxic content appearing between checkpoints may be missed. k=5 offers a good balance: 3.8 classifier calls with F1=0.188 and 84.8% token savings.

7.2.14 Impact of Error Rate Parameters. With $\pi = 0.5$ (no prior adjustment), the classifier never declares UNSAFE for strict settings ($\alpha \leq 0.05$), producing FNR=1.0. Even at $\alpha = 0.20$, FNR remains 82.3%. With $\pi = 0.01$, the adjusted thresholds become very permissive (e.g., $A_{\text{adj}} = -1.70$), causing FPR=85–100%. The best balanced operating point is (α=0.01, β=0.05, π=0.01) with F1=0.384, FPR=20.7%, FNR=11.4%—but this still has high FPR for practical deployment.

Table 17. Error Rate Parameter Ablation (CivilComments, both $\pi = 0.5$ and $\pi = 0.01$)

(α, β)	Prior π	A_{adj}	B_{adj}	FPR	FNR	AvgTok
(0.01, 0.05)	0.50	4.55	−2.99	0.000	1.000	122.1
(0.01, 0.05)	0.01	−0.04	−7.58	0.207	0.114	101.2
(0.05, 0.10)	0.50	2.89	−2.25	0.000	1.000	73.6
(0.05, 0.10)	0.01	−1.70	−6.85	0.853	0.007	25.8
(0.10, 0.15)	0.50	2.14	−1.79	0.001	0.943	40.9
(0.10, 0.15)	0.01	−2.46	−6.39	0.990	0.000	6.4
(0.20, 0.20)	0.50	1.39	−1.39	0.006	0.823	20.7
(0.20, 0.20)	0.01	−3.21	−5.98	0.999	0.000	5.1

These results confirm that the prior π is the dominant factor in system behavior, far more impactful than the choice of (α, β).

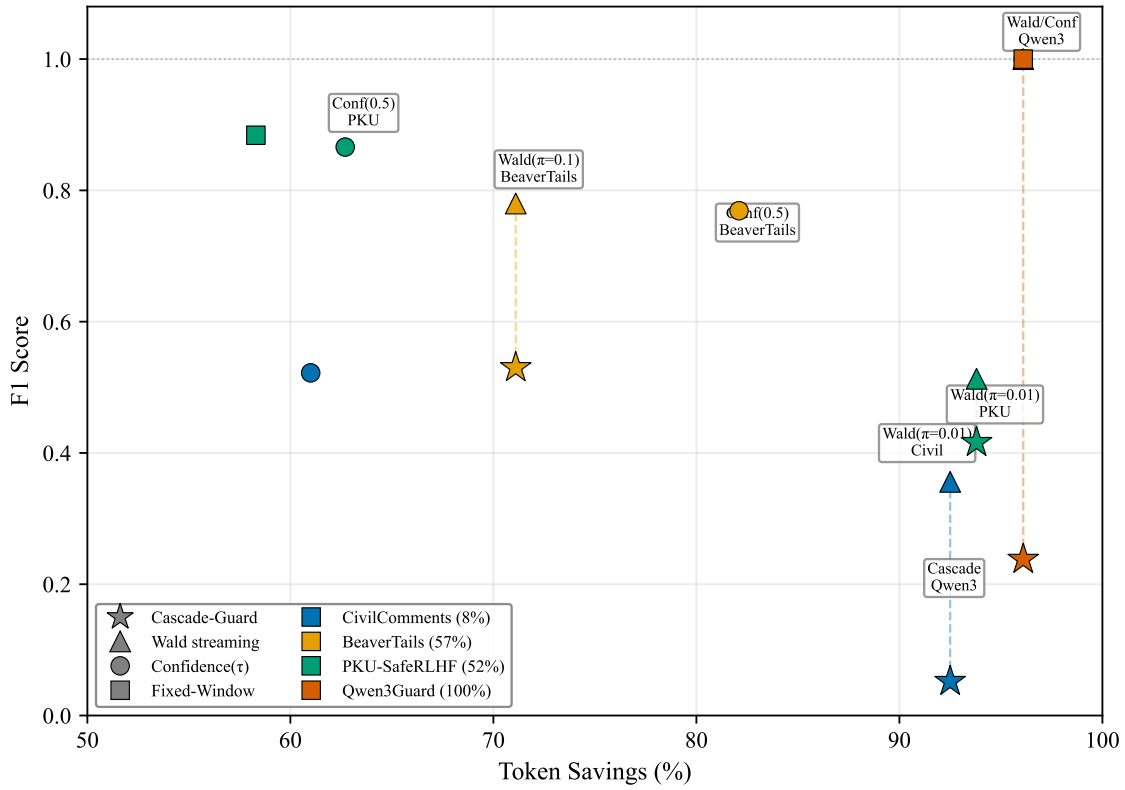

Fig. 1. F1 vs. Token Savings trade-off. Cascade (stars) achieves lower FPR than single-stage methods at the same savings level, but at F1 cost: Wald (triangles), Confidence (circles), and Fixed-Window (squares).

7.2.15 Latency-Quality Trade-off Analysis. The cascade's value depends critically on the dataset. On BeaverTails, single-stage Wald(π=0.1) achieves F1=0.780 with FPR=38.5%, and Cascade-Guard reduces FPR by 56% (38.5%→16.8%) via

Llama Guard confirmation. However, F1 drops to 0.530 because Llama Guard overrules many correct Stage 1 decisions on short prefixes. On PKU-SafeRLHF, single-stage Wald(π=0.01) achieves F1=0.513 with FPR=10.1%, and the cascade reduces FPR by 50% (10.1%→5.1%) but F1 drops to 0.416. On Qwen3GuardTest, all single-stage methods achieve perfect detection (F1=1.0) with 96.1% savings—fine-tuned RoBERTa easily detects adversarial patterns—while the cascade is the only method that fails (F1=0.238). On CivilComments (8% toxicity), Llama Guard achieves only F1=0.138 on complete text, and the cascade cannot improve upon this ceiling: Cascade(π=0.01) achieves F1=0.052 with FPR=0.2%, versus Wald streaming's F1=0.356 with FPR=4.5%.

Overall, Cascade-Guard is most valuable when the toxicity rate is moderate-to-high (>20%), or when low FPR is a hard deployment constraint that requires Llama Guard's precision.

8 Discussion

8.1 When Should Practitioners Use This Method?

The heterogeneous cascade is recommended for balanced or high-toxicity environments (>20% toxicity), where Cascade-Guard can reduce FPR by 50–96% over single-stage Wald streaming, though at F1 cost. It is also suitable when low FPR is a hard deployment constraint: the cascade's Llama Guard Stage 2 confirmation achieves near-zero FPR (0.2–2.0%). When Stage 2 is unavailable, Wald-parameterized streaming alone achieves 71–96% token savings with competitive F1. When the toxicity rate is known, setting $\pi \approx$ toxicity rate provides near-optimal Stage 1 screening with zero tuning.

The cascade is not recommended for datasets where Llama Guard's full-text F1 is already low (<0.2): the cascade cannot exceed Stage 2's capability ceiling (CivilComments: FT-LG F1=0.138, Cascade F1=0.052). It is also not suitable for highly imbalanced datasets (<10% toxicity) where both Wald streaming and cascade struggle with the rare class, or for applications requiring both high F1 and low FPR simultaneously.

8.2 Practical Deployment Workflow

For practical deployment, we recommend the following workflow. First, estimate the toxicity rate $\hat{\pi}$ from historical data. Second, set $\pi = \hat{\pi}$ and $(\alpha, \beta) = (0.05, 0.10)$ as defaults. Third, enable the cascade with Llama Guard as Stage 2 confirmation. Fourth, evaluate on a held-out validation set and measure empirical FPR and FNR. Fifth, if FPR is too high, decrease π or increase α; if FNR is too high, decrease π. For adversarial robustness, use $\pi \leq 0.05$ regardless of observed toxicity rate.

8.3 Limitations

(1) Wald's error bounds do not hold when using neural classifier outputs and prior adjustment. The cascade empirically reduces FPR through model diversity but does not provide guaranteed error control.
(2) Performance is highly sensitive to π. We provide guidelines ($\pi \approx$ toxicity rate), but incorrect settings degrade performance significantly.
(3) While we validated end-to-end latency with an integrated LLM generation loop on RTX 4090 (Section 7.2.12), the streaming detection was evaluated on pre-existing text datasets rather than during live LLM generation with feedback between detection and generation. The measured latency overhead (Table 13) confirms the analytical model is conservative, but live deployment may introduce additional overhead from context switching and batch scheduling.

(4) RoBERTa re-encodes the entire prefix at each step (no KV-cache). For long sequences, chunking is essential to maintain efficiency.
(5) Llama Guard adds ~86ms latency for flagged samples. When Stage 1 triggers frequently (high Stage 1 FPR), this overhead is substantial. The cascade is most efficient when the Stage 1 trigger rate is moderate (30–60%).
(6) The architecture requires loading both RoBERTa (125M params) and Llama Guard (8B params), necessitating sufficient GPU memory. On resource-constrained devices, a smaller Stage 2 model may be needed.
(7) While model diversity is the cascade's key strength, Llama Guard may have failure modes different from RoBERTa's. For example, Llama Guard may be overly conservative on certain content types, or its zero-shot classification may not align perfectly with domain-specific definitions of toxicity. The cascade inherits both models' limitations.
(8) Llama Guard classifies the prefix at the stopping point, which may be shorter than the full output. Prefix-level safety judgment may differ from full-text judgment, especially for late-toxicity content.

8.4 Sensitivity to Distribution Shifts

A practical concern for deployment is how the system performs when the true toxicity rate π^* changes over time, since the prior π is set at configuration time.

We evaluate each Wald-parameterized configuration across multiple datasets with different toxicity rates (Table 7). The same (α=0.05, β=0.10, π=0.01) configuration produces F1 ranging from 0.356 (CivilComments, π^*=8%) to 1.000 (Qwen3GuardTest, π^*=100%). This variability is expected: π=0.01 is calibrated for low-toxicity environments and produces aggressive screening on balanced datasets.

An important property is that the relationship between π and error rates is monotonic (Table 9): decreasing π always increases recall and FPR. This means practitioners can adjust π at deployment time based on observed metrics without retraining:

- If observed FPR is too high $\Rightarrow$ increase π (more conservative screening)
- If observed FNR is too high $\Rightarrow$ decrease π (more aggressive screening)

As a practical guideline for distribution shifts: monitor empirical FPR over a rolling window. If FPR exceeds 2α, increase π by a factor of 2. For sudden distribution shifts (e.g., coordinated attack causing toxicity to spike from 5% to 50%), we recommend setting $\pi \leq 0.05$ as a conservative default, since lower priors guarantee higher recall at the cost of more Stage 2 confirmations. The cascade's Stage 2 confirmation mitigates the FPR increase from an aggressive prior.

9 Conclusion

We presented an empirical study of heterogeneous two-stage cascading for streaming LLM safety detection, asking: *when does adding a second, more powerful model actually help?*

Stage 1 uses a fine-tuned RoBERTa with Wald-formula-derived thresholds parameterized by (α, β, π) to achieve aggressive screening during streaming generation. Stage 2 applies Llama Guard, an 8B-parameter generative safety model, to confirm only the flagged prefixes, intended to eliminate false alarms through its independent decision boundary.

We have been explicit that the single-stage Wald method is mathematically equivalent to confidence thresholding—a single classifier cannot escape the precision-recall trade-off regardless of threshold choice. The heterogeneous two-stage architecture produces genuinely different decision boundaries because RoBERTa and Llama Guard have different

architectures (encoder vs. generative), training objectives, and inductive biases. However, experiments on four diverse safety benchmarks reveal a mixed picture. On balanced datasets (BeaverTails, PKU-SafeRLHF), the cascade achieves 50–56% FPR reduction over single-stage Wald streaming, but F1 decreases because Llama Guard, applied on short prefixes, overrules many correct Stage 1 decisions. On Qwen3GuardTest (100% adversarial), all single-stage methods achieve perfect detection (F1=1.0) with 96.1% token savings, while the zero-shot cascade fails (F1=0.238). Prefix-aware fine-tuning substantially improves the cascade (F1=0.736, +209%), but it still falls short of single-stage perfection. On low-toxicity datasets (CivilComments, 8%), neither Wald streaming nor cascade is competitive—Llama Guard's base F1=0.138 creates an insurmountable ceiling.

The architecture's main limitation is the requirement to load two models (125M + 8B parameters), though the latency cost of Stage 2 is amortized over non-flagged samples. More fundamentally, cascade effectiveness is bounded by Stage 2's prefix-level performance: when Llama Guard overrules correct Stage 1 decisions on short prefixes, the cascade hurts rather than helps. Prefix-aware fine-tuning of Stage 2 partially resolves this bottleneck (Section 7.2.4), validating the cascade architecture while revealing remaining limitations. Future work includes: (1) adaptive Stage 1 thresholds that account for Stage 2's known error profile, (2) evaluating other Stage 2 candidates with stronger prefix-level performance, and (3) live deployment evaluation under production workloads with feedback between detection and generation.

References

[1] Abraham Wald. Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2):117–186, 1945.

[2] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning*, 1321–1330, 2017.

[3] T.L. Lai. Sequential analysis: some classical problems and new challenges. *Statistica Sinica*, 11(2):303–408, 2001.

[4] Francesco Orabona, Andrea Locatello, Andrea Passerini, et al. Online Learning with Sequential Hypothesis Testing. In *Proceedings of the 35th Conference on Learning Theory (COLT)*, 2022.

[5] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2022.

[6] Jiaming Ji, Donghai Hong, Borong Zhang, Boyuan Chen, and Josef Dai. BeaverTails: Towards Improved Safety Alignment of LLM via a Human-Preference Dataset. In *Advances in Neural Information Processing Systems*, 2023.

[7] Jiaming Ji, Mickel Liu, Haiyang Lin, Xincheng Zhang, and Qiang Guo. PKU-SafeRLHF: Preference Learning for Safety Alignment. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[8] Qwen Team, Alibaba Group. Qwen3GuardTest: A Benchmark for LLM Guardrail Testing with Position Annotations. https://huggingface.co/datasets/Qwen/Qwen3GuardTest, 2024.

[9] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, et al. Llama Guard: LLM-based Input/Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674*, 2023.

[10] OpenAI. GPT-4 System Card. https://cdn.openai.com/papers/gpt-4-system-card.pdf, 2023.

[11] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*, 2019.

[12] Google Jigsaw. Perspective API: Machine Learning for Content Moderation. https://perspectiveapi.com/, 2024.

[13] Samuel Gehman, Suchin Gururangan, Maarten Sap, Yejin Choi, and Noah A. Smith. RealToxicityPrompts: Evaluating Neural Toxic Degeneration in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

[14] Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, et al. ToxiGen: A Large-Scale Machine-Generated Dataset for Adversarial and Implicit Hate Speech Detection. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.

[15] Aaron Grattafiori, Abhimanyu Dubey, et al. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.

[16] Xiaohui Chen and Yutong Wang. Online Detection of LLM-Generated Texts via Sequential Hypothesis Testing by Betting. *arXiv preprint arXiv:2410.22318*, 2024.

[17] HaseenaRahmath K.P. and Pratistha Srivastava. Early-Exit Deep Neural Network: A Comprehensive Survey. *ACM Computing Surveys*, DOI:10.1145/3698767, 2024.

[18] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Sayan Mukherjee, and Lili Mu. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations*, 2024.

[19] Caio Surace, Gabriel Synnaeve, and Szymon Sidor. Early Exit: A Review of Methods for Adaptive Inference in Deep Neural Networks. *arXiv preprint arXiv:2301.04829*, 2023.

[20] Paul Viola and Michael J. Jones. Robust Real-Time Face Detection. *International Journal of Computer Vision*, 57(2):137–154, 2004.